VISUALIZATION

Visualization refers to the process of presenting information in a visual format such as charts, graphs, and diagrams. In the context of data analysis, data visualization means converting raw data into visual representations that make it easier to understand patterns, trends, and insights.

Instead of analysing large tables of numbers, visualization helps users quickly grasp important information. It plays a crucial role in decision-making, reporting, and storytelling with data.

Python Libraries for Visualization

Python provides several powerful libraries for data visualization, including:

- Matplotlib
- Seaborn
- Plotly
- Pandas Visualization Tools
- Bokeh

Among these, Matplotlib and Seaborn are widely used and form the foundation of most visualizations.

MATPLOTLIB

Matplotlib is one of the oldest and most widely used data visualization libraries in Python. It was developed by John Hunter in 2002. It is mainly used for creating 2D plots, although it also supports limited 3D plotting.

It works well with NumPy arrays and provides complete control over every element in a graph.

Components of a Matplotlib Plot

1. Figure

A figure is the outer container that holds one or more plots. It acts like a canvas where all visual elements are drawn.

2. Axes

Axes represent the actual plotting area where data is displayed. A figure can contain multiple axes.

3. Axis

Axis refers to the horizontal (x-axis) and vertical (y-axis) lines that define the coordinate system. It also handles tick marks and labels.

4. Artists

Artists are all visual elements in a plot, including lines, text, labels, shapes, etc.

Pyplot Module

Pyplot is a submodule of Matplotlib that provides a simple interface for creating plots. It contains functions for:

- Creating figures
- Plotting data
- Adding labels and titles
- Displaying graphs

Example import:

import matplotlib.pyplot as plt

import numpy as np

Types of Plots in Matplotlib:

1. LINE CHART

A line chart connects data points using straight lines. It is useful for showing trends over time.

Key Features:

- Displays continuous data
- Easy to understand patterns

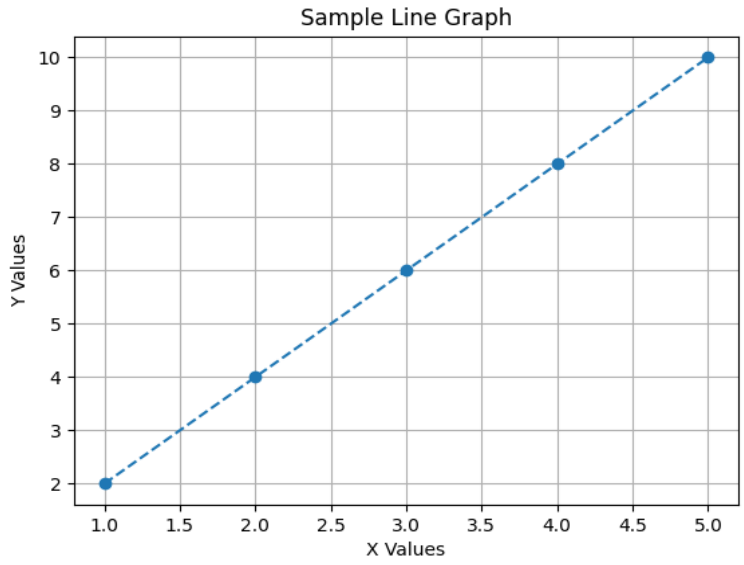
Functions Used:

- plot() → creates the line
- title() → sets title
- xlabel() / ylabel() → axis labels
- show() → displays graph

Code snippet:

```
import matplotlib.pyplot as plt
x = list(range(1, 6))
y = [2, 4, 6, 8, 10]
plt.plot(x, y, marker='o', linestyle='--')
plt.title("Sample Line Graph")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.grid(True)
plt.show()
```

Output:



2. BAR CHART

Bar charts use rectangular bars to represent values. The height of each bar indicates the magnitude of data.

Applications:

- Comparing categories
- Visualizing discrete data

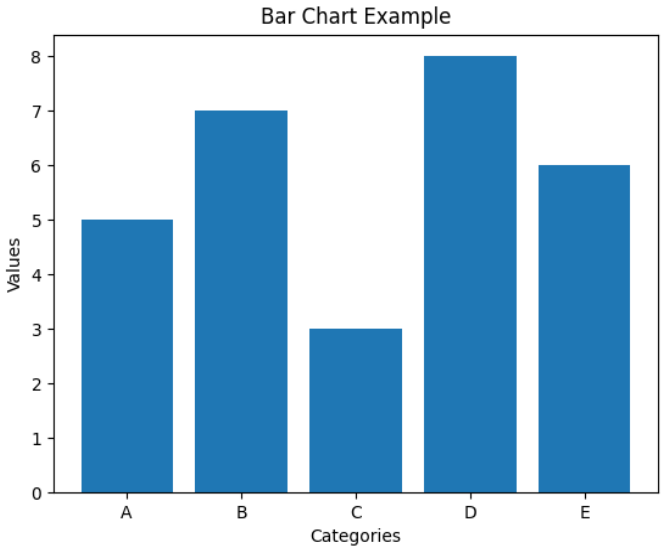
Function:

- bar(x, y, color, width)

Code snippet:

```
import matplotlib.pyplot as plt
categories = ["A", "B", "C", "D", "E"]
values = [5, 7, 3, 8, 6]
plt.bar(categories, values)
plt.title("Bar Chart Example")
plt.xlabel("Categories")
plt.ylabel("Values")
plt.show()
```

Output:



3. HISTOGRAM

A histogram shows the distribution of numerical data by grouping values into intervals (bins).

Key Points:

- Displays frequency distribution
- Helps identify data spread and skewness

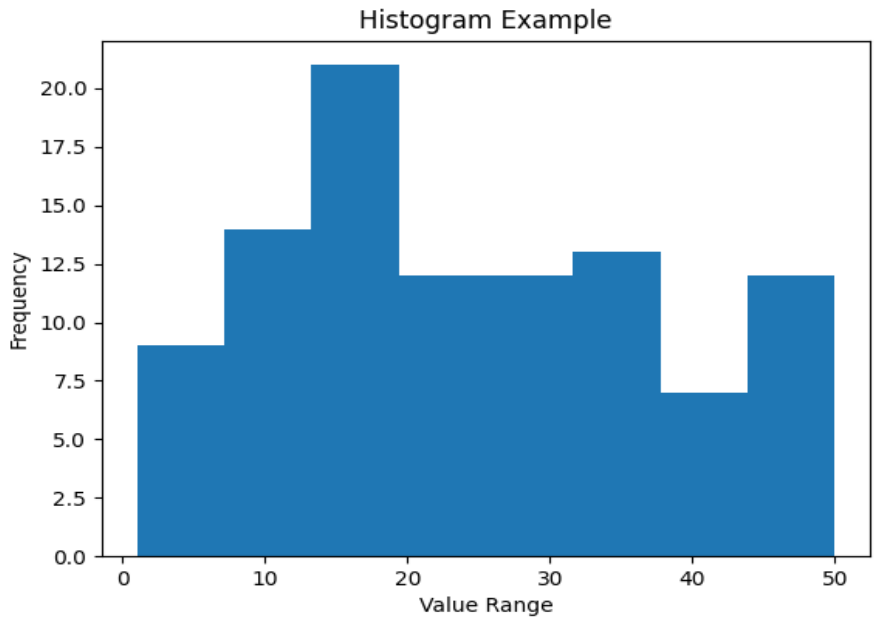
Function:

- hist(x, bins, color, edgecolor)

Code snippet:

```
import matplotlib.pyplot as plt
import random
data = [random.randint(1, 50) for i in range(100)]
plt.hist(data, bins=8)
plt.title("Histogram Example")
plt.xlabel("Value Range")
plt.ylabel("Frequency")
plt.show()
```

Output:



4. SCATTER PLOT

Scatter plots use points to represent the relationship between two variables.

Uses:

- Identifying correlation
- Detecting clusters or outliers

Function:

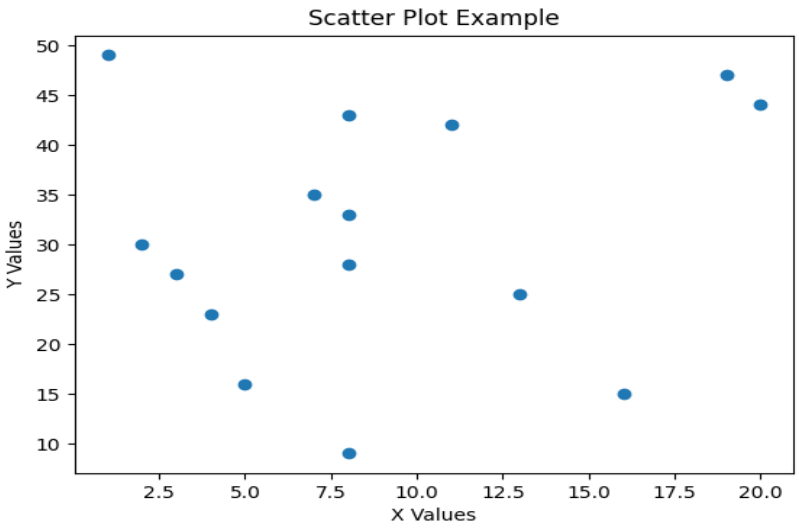
- scatter(x, y, color)

Multiple datasets can be plotted using different colors and legends.

Code snippet:

```
import matplotlib.pyplot as plt
import random
x = [random.randint(1, 20) for i in range(15)]
y = [random.randint(1, 50) for i in range(15)]
plt.scatter(x, y)
plt.title("Scatter Plot Example")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show()
```

Output:



5. PIE CHART

Pie charts represent data as slices of a circle, where each slice corresponds to a percentage.

Best For:

- Showing proportions
- Comparing parts of a whole

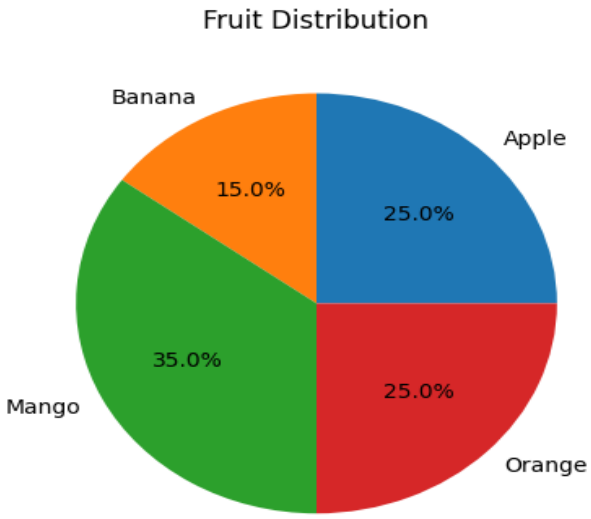
Features:

- Labels
- Colors
- Exploded slices

Code snippet:

```
import matplotlib.pyplot as plt
labels = ["Apple", "Banana", "Mango", "Orange"]
sizes = [25, 15, 35, 25]
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Fruit Distribution")
plt.show()
```

Output:



SEABORN

Seaborn is a high-level visualization library built on top of Matplotlib. It simplifies complex plotting tasks and provides better default styles and colour themes.

It is especially useful for statistical data visualization.

Features of Seaborn

- Attractive default designs
- Built-in colour palettes
- Simplified syntax
- Direct integration with Pandas DataFrames
- Advanced statistical plots

Categories of Plots in Seaborn

1. Relational Plots

Used to show relationships between variables.

- scatterplot()
- lineplot()
- relplot()

2. Categorical Plots

Used for categorical data comparison.

- barplot()
- countplot()
- boxplot()
- violinplot()
- swarmplot()
- pointplot()

3. Distribution Plots

Used to analyze data distribution.

- histplot()
- kdeplot()

- rugplot()
- distplot()

4. Regression Plots

Used to show relationships with regression lines.

- regplot()
- lmplot()

5. Matrix Plots

Used for correlation and clustering.

- heatmap()
- clustermap()

Types of Plots in Seaborn:

1. LINEPLOT:

A line plot in Seaborn is used to visualize the relationship between two variables, typically showing how one variable changes with respect to another. It is especially useful for displaying trends over time or continuous data.

Key Features:

- Displays trends and patterns clearly
- Supports multiple lines using categories

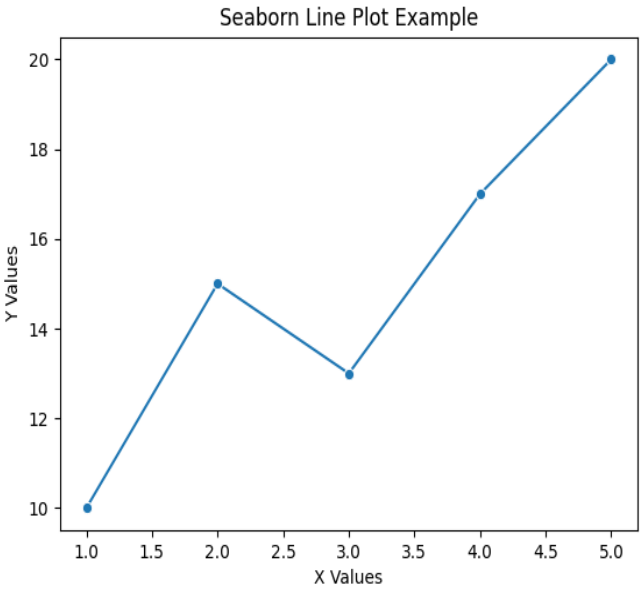
Common Parameters:

- x → values for x-axis
- y → values for y-axis
- hue → grouping variable
- style → different line styles
- markers → shows markers on data points

Code snippet:

```python
import seaborn as sns
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 15, 13, 17, 20]
sns.lineplot(x=x, y=y, marker='o')
plt.title("Seaborn Line Plot Example")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show()
```

Output:



2. BOXPLOT:

A box plot (or box-and-whisker plot) in Seaborn is used to represent the distribution of numerical data through quartiles. It provides a summary of the dataset by showing the minimum value, first quartile (Q1), median (Q2), third quartile (Q3), and maximum value.

Components of a Box Plot:

- **Box** → Represents the interquartile range (Q1 to Q3)
- **Median line** → Shows the middle value (Q2)
- **Whiskers** → Extend to minimum and maximum values (excluding outliers)
- **Outliers** → Data points outside whiskers, shown as individual dots

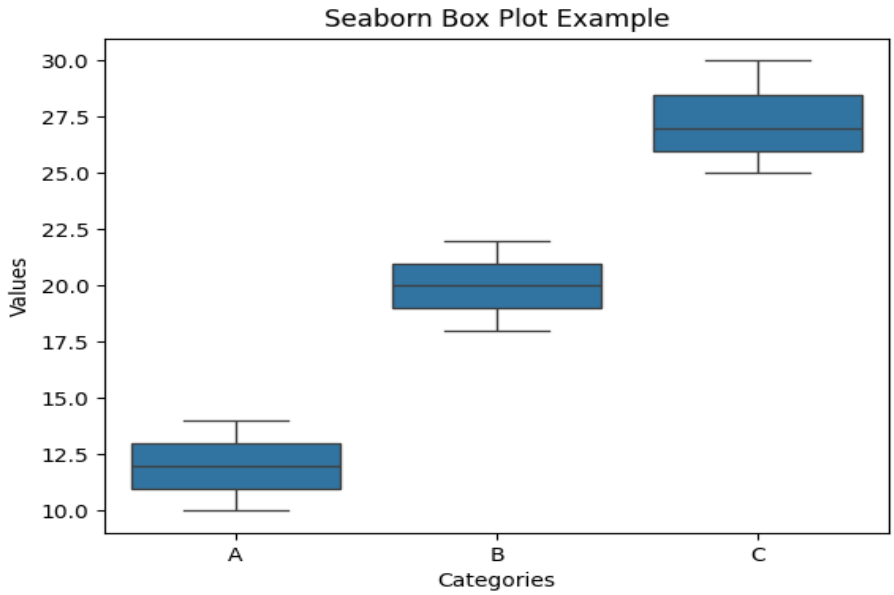
Key Features:

- Easy comparison between different categories.
- Detects skewness and outliers.
- Works well with categorical and numerical data.

Code snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
categories = ["A", "A", "A", "B", "B", "B", "C", "C", "C"]
values = [10, 12, 14, 18, 20, 22, 25, 27, 30]
sns.boxplot(x=categories, y=values)
plt.title("Seaborn Box Plot Example")
plt.xlabel("Categories")
plt.ylabel("Values")
plt.show()
```

Output:



3. DISTPLOT:

A distplot (distribution plot) in Seaborn is used to visualize how data is distributed. It combines a histogram (to show frequency) and a density curve (KDE – Kernel Density Estimate) to give a smooth representation of the data distribution.

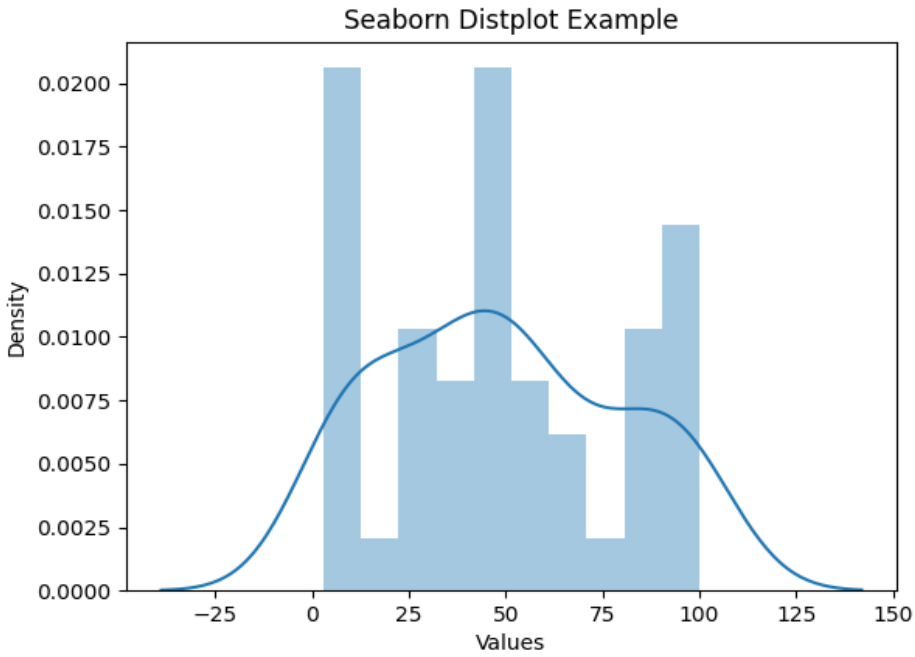
Key Features:

- Shows both histogram and density curve together
- Helps in identifying normal or skewed distribution
- Easy to visualize large datasets

Code snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
import random
data = [random.randint(1, 100) for i in range(50)]
sns.distplot(data, bins=10)
plt.title("Seaborn Distplot Example")
plt.xlabel("Values")
plt.ylabel("Density")
plt.show()
```

Output:



4. REGPLOT:

A regression plot (regplot) in Seaborn is used to visualize the relationship between two numerical variables along with a regression line (best-fit line). It helps in understanding how one variable changes with respect to another and whether there is a positive, negative, or no correlation between them.

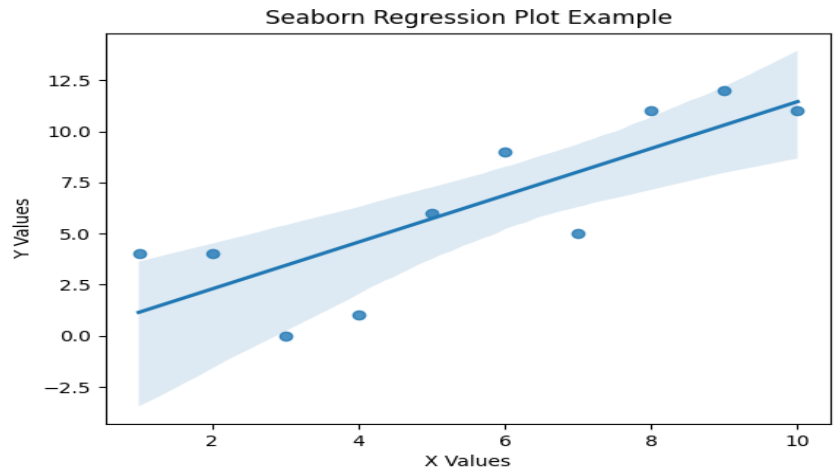
Key Features:

- Combines **scatter plot + regression line**
- Helps in identifying correlation between variables
- Useful in predictive analysis and trend estimation
- Automatically fits a linear regression model

Code snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
import random
x = [i for i in range(1, 11)]
y = [value + random.randint(-3, 3) for value in x]
sns.regplot(x=x, y=y)
plt.title("Seaborn Regression Plot Example")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show()
```

Output:



5. HEATMAP:

A heatmap in Seaborn is a graphical representation of data where values are displayed as colours. It is mainly used to visualize matrix data or correlation between variables in a dataset.

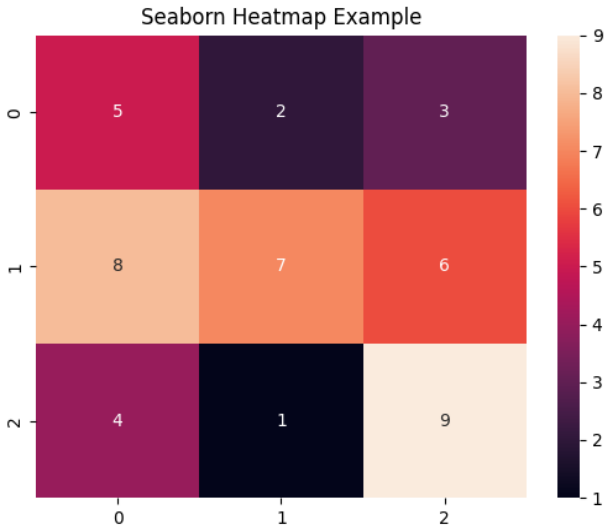
Key Features:

- Displays data in a **color-coded format**
- Useful for **correlation matrices**
- Helps in identifying **strong and weak relationships**
- Easy to interpret large datasets

Code snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
data = [
    [5, 2, 3],
    [8, 7, 6],
    [4, 1, 9]
]
sns.heatmap(data, annot=True)
plt.title("Seaborn Heatmap Example")
plt.show()
```

Output:



PLOTLY

Plotly is a powerful data visualization library in Python used to create interactive and web-based graphs. It allows users to build dynamic visualizations with features like zooming, hovering, and animations.

Plotly is widely used in data analysis, dashboards, and web applications because of its ability to create highly interactive and visually appealing charts.

Features of Plotly

- Interactive graphs (zoom, pan, hover)
- Supports a wide variety of charts
- Works well with **Pandas DataFrames**
- Can be used in web frameworks like Dash
- Provides both **high-level (Plotly Express)** and **low-level (Graph Objects)** interfaces

Categories of Plots in Plotly:

1. Statistical Charts

- Histogram
- Box Plot
- Violin Plot
- Density Plot

2. Financial Charts

- Candlestick Chart
- OHLC Chart

3. Hierarchical Charts

- Treemap
- Sunburst Chart
- Icicle Chart

6. Map-Based Charts

- Choropleth Map
- Bubble Map

7. Advanced Charts

- Heatmap
- Funnel Chart
- Waterfall Chart
- Radar Chart

Types of Plots in Plotly:

1. BOXPLOT:

A box plot (also known as a box-and-whisker plot) in Plotly is used to display the distribution of numerical data based on five key summary values:

- Minimum value
- First quartile (Q1)
- Median (Q2)
- Third quartile (Q3)
- Maximum value

It is very useful for understanding the spread and variation of data and for identifying outliers.

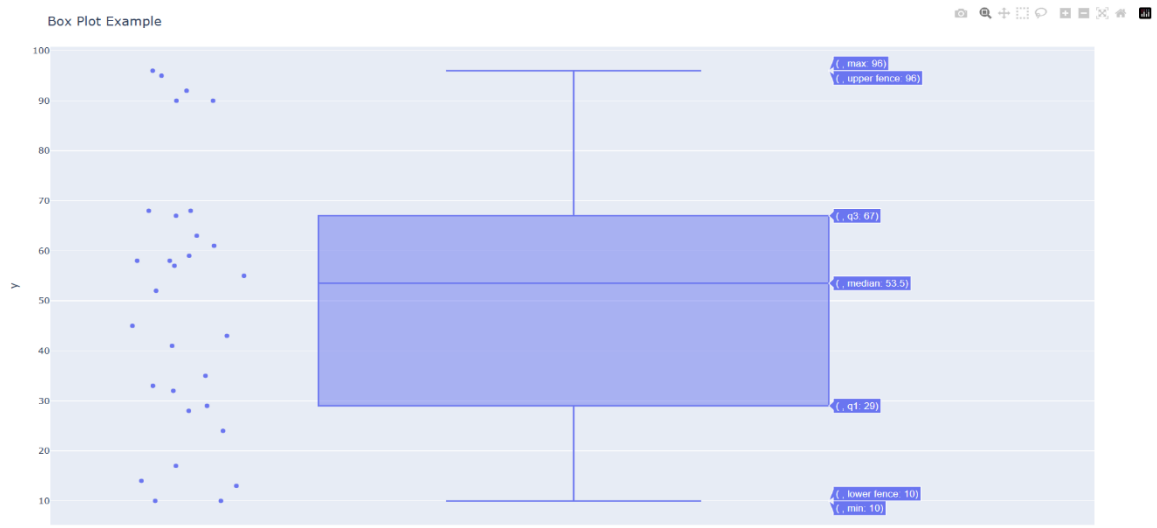
Key Features:

- Clearly shows data distribution
- Helps compare multiple datasets
- Detects outliers easily
- Interactive in Plotly (hover, zoom)

Code snippet:

```
import plotly.express as px
import random
data = [random.randint(10, 100) for i in range(30)]
fig = px.box(y=data, points="all")
fig.update_layout(title="Box Plot Example")
fig.show()
```

Output:



2. CANDLESTICK CHART:

A candlestick chart in Plotly is a type of financial chart used to represent the price movement of stocks, cryptocurrencies, or other assets over a period of time. It displays four important values for each time interval:

- **Open price** → price at the beginning
- **Close price** → price at the end
- **High price** → highest value reached
- **Low price** → lowest value reached

Each data point is shown as a candlestick, which consists of:

- A body → represents the range between open and close
- Wicks (shadows) → show high and low values

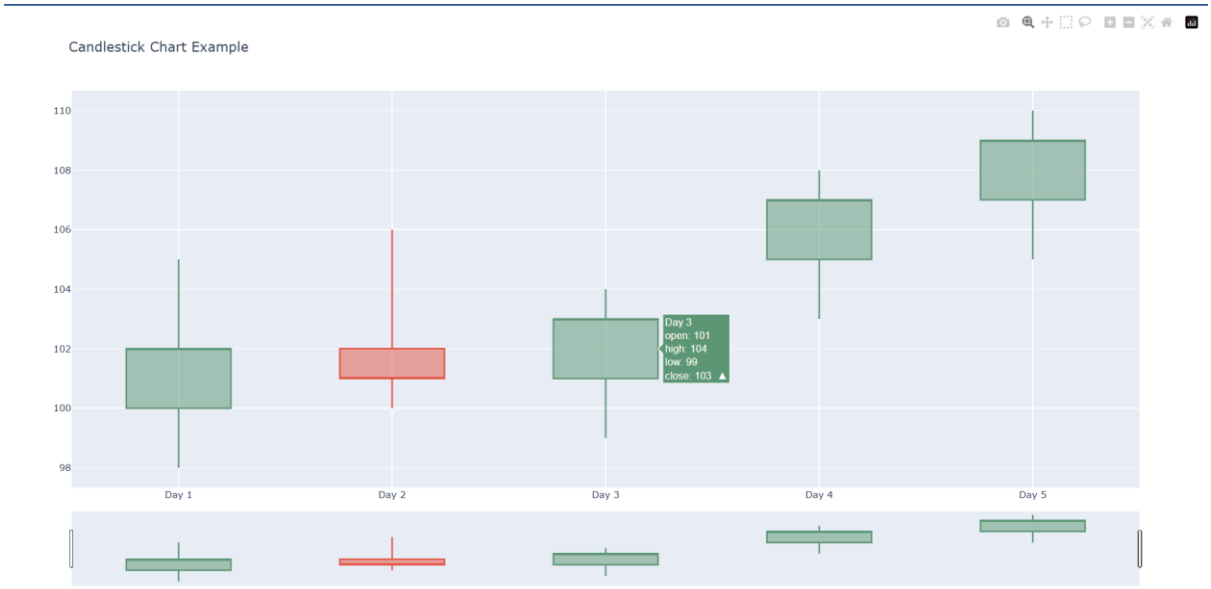
Key Features:

- Shows detailed price movement
- Useful for financial and stock analysis
- Helps identify trends and patterns
- Interactive in Plotly (zoom, hover, etc.)

Code snippet:

```
import plotly.graph_objects as go
x = ["Day 1", "Day 2", "Day 3", "Day 4", "Day 5"]
open_price = [100, 102, 101, 105, 107]
high_price = [105, 106, 104, 108, 110]
low_price = [98, 100, 99, 103, 105]
close_price = [102, 101, 103, 107, 109]
fig = go.Figure(data=[go.Candlestick(
    x=x,
    open=open_price,
    high=high_price,
    low=low_price,
    close=close_price
)])
fig.update_layout(title="Candlestick Chart Example")
fig.show()
```

Output:



3. SUNBURST CHART:

A sunburst chart in Plotly is used to represent hierarchical data in a circular (radial) layout. It is similar to a treemap, but instead of rectangles, it uses concentric rings to display different levels of the hierarchy.

- The center (inner circle) represents the root or main category
- Outer rings represent subcategories
- Each segment's size corresponds to its value

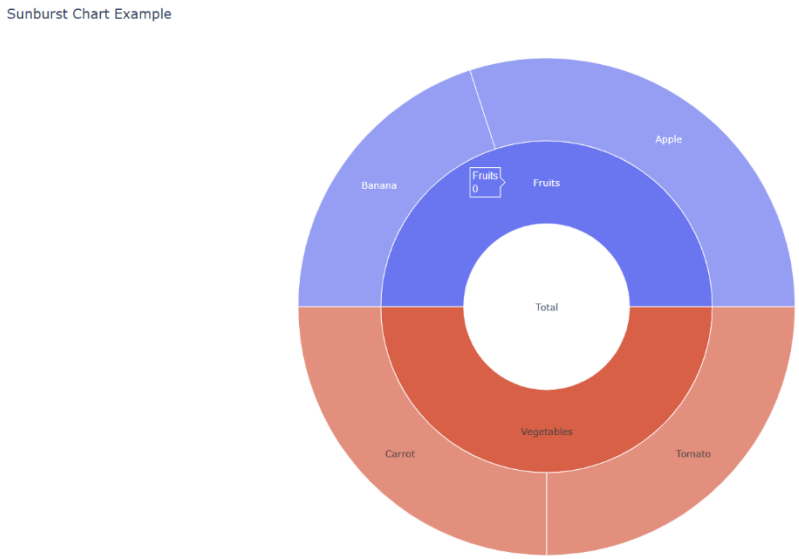
Key Features:

- Displays hierarchical data in a circular format
- Easy to visualize parent-child relationships
- Interactive (click to zoom into levels)
- Visually appealing and space-efficient

Code snippet:

```
import plotly.graph_objects as go
labels = ["Total", "Fruits", "Vegetables", "Apple", "Banana", "Carrot", "Tomato"]
parents = ["", "Total", "Total", "Fruits", "Fruits", "Vegetables", "Vegetables"]
values = [0, 0, 0, 30, 20, 25, 25]
fig = go.Figure(go.Sunburst(
    labels=labels,
    parents=parents,
    values=values
))
fig.update_layout(title="Sunburst Chart Example")
fig.show()
```

Output:



4. BUBBLE CHART

A bubble map in Plotly is a geographical visualization where data points are plotted on a map using circles (bubbles). The size of each bubble represents a numerical value, making it easy to compare data across different locations.

It is commonly used for:

- Population visualization
- Sales distribution across regions
- Comparing values across cities or countries

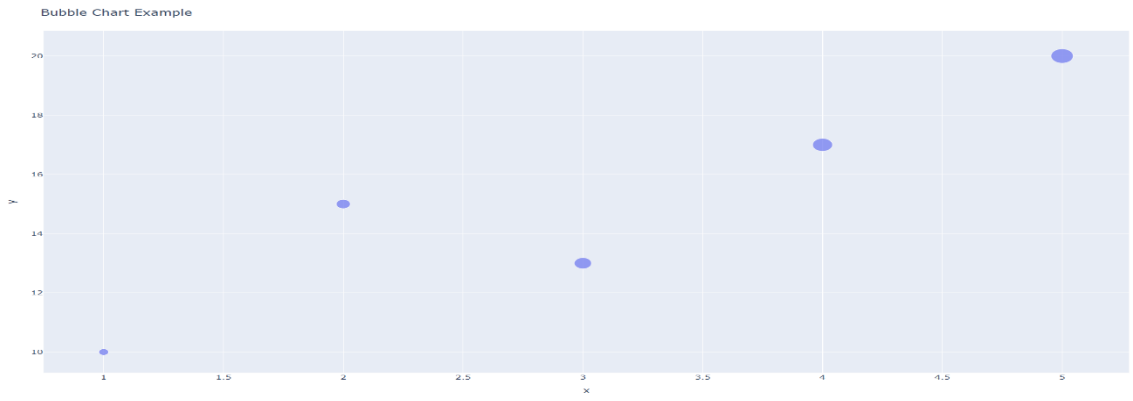
Key Features:

- Displays data on a geographical map
- Bubble size represents magnitude of values
- Interactive (zoom, hover, pan)
- Can show additional information using labels

Code snippet:

```
import plotly.express as px
x = [1, 2, 3, 4, 5]
y = [10, 15, 13, 17, 20]
sizes = [20, 40, 60, 80, 100]
fig = px.scatter(x=x, y=y, size=sizes)
fig.update_layout(title="Bubble Chart Example")
fig.show()
```

Output:



5. RADAR CHART:

A radar chart (also called a spider chart) in Plotly is used to display multivariate data in a circular form. Each variable is represented as an axis starting from the center, and the values are plotted along these axes and connected to form a shape.

Radar charts are useful for:

- Comparing multiple variables for a single entity
- Comparing multiple entities across the same set of variables
- Performance analysis (e.g., skills, features, scores)

Key Features:

- Circular layout with multiple axes
- Easy comparison of multiple variables
- Visually appealing and interactive in Plotly
- Helps identify strengths and weaknesses

Code snippet:

```
import plotly.graph_objects as go
categories = ["Math", "Science", "English", "History", "Sports"]
values = [80, 75, 85, 70, 90]
fig = go.Figure()

fig.add_trace(go.Scatterpolar(
    r=values,
    theta=categories,
    fill='toself',
    name='Student Performance'
))
fig.update_layout(title="Radar Chart Example")
fig.show()
```

Output:

BOKEH

Bokeh is a powerful data visualization library in Python used to create interactive and web-based visualizations. It is especially useful for building interactive dashboards and real-time applications that run in the web browsers.

Unlike static libraries, Bokeh allows users to interact with graphs using tools like zooming, panning, and hovering.

Features of Bokeh

- Interactive plots (zoom, pan, hover, selection)
- Supports real-time and streaming data
- Can create dashboards and web applications
- High-performance rendering for large datasets
- Integrates with tools like Pandas and NumPy

Variety of graphs in bokeh:

1. Basic Plots

These are the most commonly used graphs:

- Line Plot (line())
- Scatter Plot (scatter())
- Circle Plot (circle())
- Square Plot (square())
- Triangle Plot (triangle())

2. Bar Charts

Used for categorical data:

- Vertical Bar Chart (vbar())
- Horizontal Bar Chart (hbar())

3. Interactive Graphs

Bokeh is known for interactivity:

- Hover-enabled plots
- Linked plots
- Selection-based plots

4. Area and Region Plots

- Area Plot (patch(), varea(), harea())
- Stacked Area Plot

5. Advanced Plots

- Heatmap
- Step Plot (step())
- Patch Plot (for polygons)
- Multi-line Plot (multi_line())

Types of Plots in Bokeh:

1. SQUARE PLOT:

A square plot in Bokeh is a type of scatter plot where data points are represented using square-shaped markers instead of circles. It is useful when you want to visualize the relationship between two variables while using a different marker style for better distinction.

Square plots are often used in:

- Comparing datasets with different marker shapes
- Highlighting specific data points
- Improving visual clarity in crowded plots

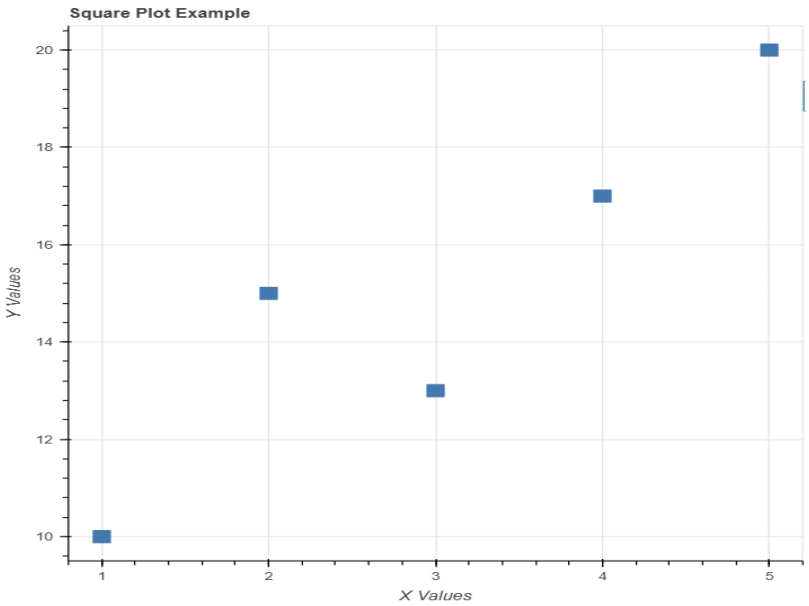
Key Features:

- Uses square markers for data points
- Simple and easy to create
- Interactive (zoom, pan, hover)
- Can be combined with other glyphs (like line, circle)

Code snippet:

```
from bokeh.plotting import figure, show
x = [1, 2, 3, 4, 5]
y = [10, 15, 13, 17, 20]
p = figure(title="Square Plot Example", x_axis_label="X Values", y_axis_label="Y Values")
p.square(x, y, size=12)
show(p)
```

Output:



2. HORIZONTAL BARCHART:

A horizontal bar chart in Bokeh is used to represent categorical data using horizontal rectangular bars. In this chart, categories are displayed on the y-axis, and their corresponding values are shown along the x-axis.

It is especially useful when:

- Category names are long and need more space
- Comparing values across multiple categories
- Displaying ranked data clearly

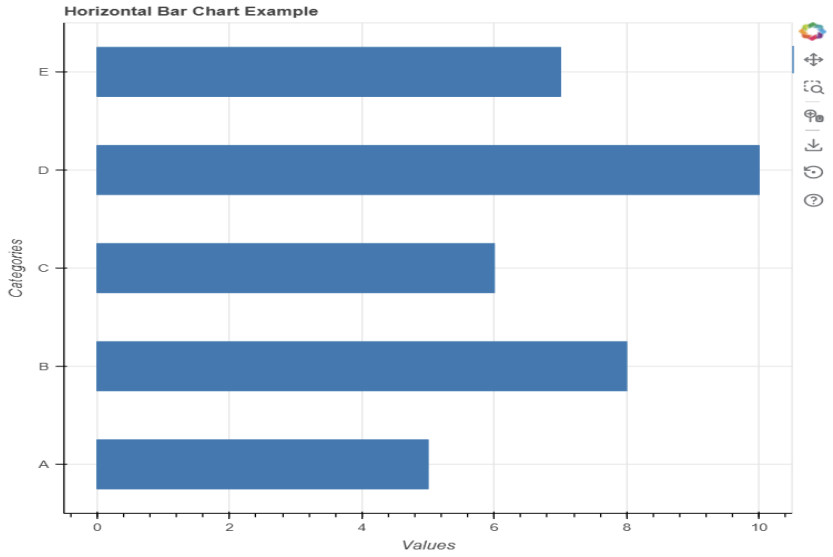
Key Features:

- Bars are drawn horizontally
- Easy to compare different categories
- Suitable for large or long category labels
- Interactive (zoom, pan, hover)

Code snippet:

```
from bokeh.plotting import figure, show
categories = ["A", "B", "C", "D", "E"]
values = [5, 8, 6, 10, 7]
p = figure(y_range=categories, title="Horizontal Bar Chart Example",
           x_axis_label="Values", y_axis_label="Categories")
p.hbar(y=categories, right=values, height=0.5)
show(p)
```

Output:



3. LINKED PLOTS:

Linked plots in Bokeh are multiple graphs that are connected through shared interactions. This means when you perform an action on one plot (like zooming, panning, or selecting data points), the same action is reflected in the other plots automatically.

Linked plots are very useful for:

- Comparing multiple datasets
- Exploring relationships between variables
- Building interactive dashboards

Key Features:

- Shared x-axis or y-axis between plots
- Synchronized zooming and panning
- Can link selections (highlighting data points)
- Improves data exploration and analysis

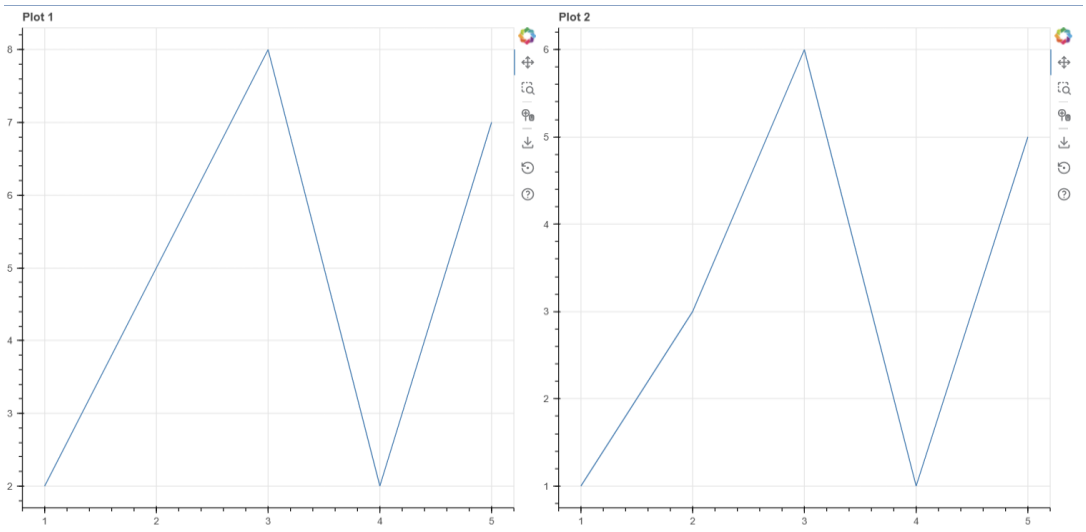
Types of Linking:

1. Linked Axes → Multiple plots share the same axis range
2. Linked Selection → Selecting data in one plot highlights it in others

Code snippet:

```
from bokeh.plotting import figure, show
from bokeh.layouts import row
x = [1, 2, 3, 4, 5]
y1 = [2, 5, 8, 2, 7]
y2 = [1, 3, 6, 1, 5]
p1 = figure(title="Plot 1")
p1.line(x, y1)
p2 = figure(title="Plot 2", x_range=p1.x_range)
p2.line(x, y2)
show(row(p1, p2))
```

Output:



4. AREA PLOT:

An area plot in Bokeh is used to visualize data where the area under a line is filled with color. It is similar to a line chart, but the region between the line and the axis is shaded, making it easier to understand the magnitude of values over a range.

Area plots are commonly used for:

- Showing trends over time
- Comparing quantities
- Visualizing cumulative data

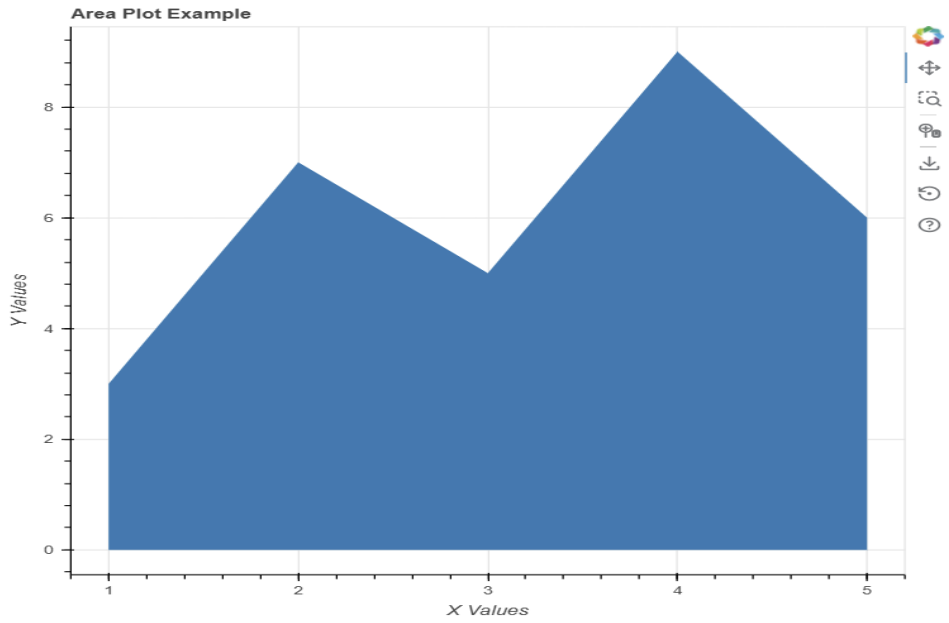
Key Features:

- Filled area under the curve
- Clearly shows magnitude and trend
- Can represent single or multiple datasets
- Interactive (zoom, pan, hover)

Code snippet:

```
from bokeh.plotting import figure, show
x = [1, 2, 3, 4, 5]
y = [3, 7, 5, 9, 6]
p = figure(title="Area Plot Example", x_axis_label="X Values", y_axis_label="Y Values")
p.varea(x=x, y1=0, y2=y)
show(p)
```

Output:



5. PATCH PLOT:

A patch plot in Bokeh is used to draw filled shapes (polygons) by connecting a set of points. It is useful for visualizing areas, regions, or custom shapes defined by coordinates.

Patch plots are commonly used for:

- Drawing geometric shapes
- Highlighting specific regions in a graph

- Creating maps or custom visualizations

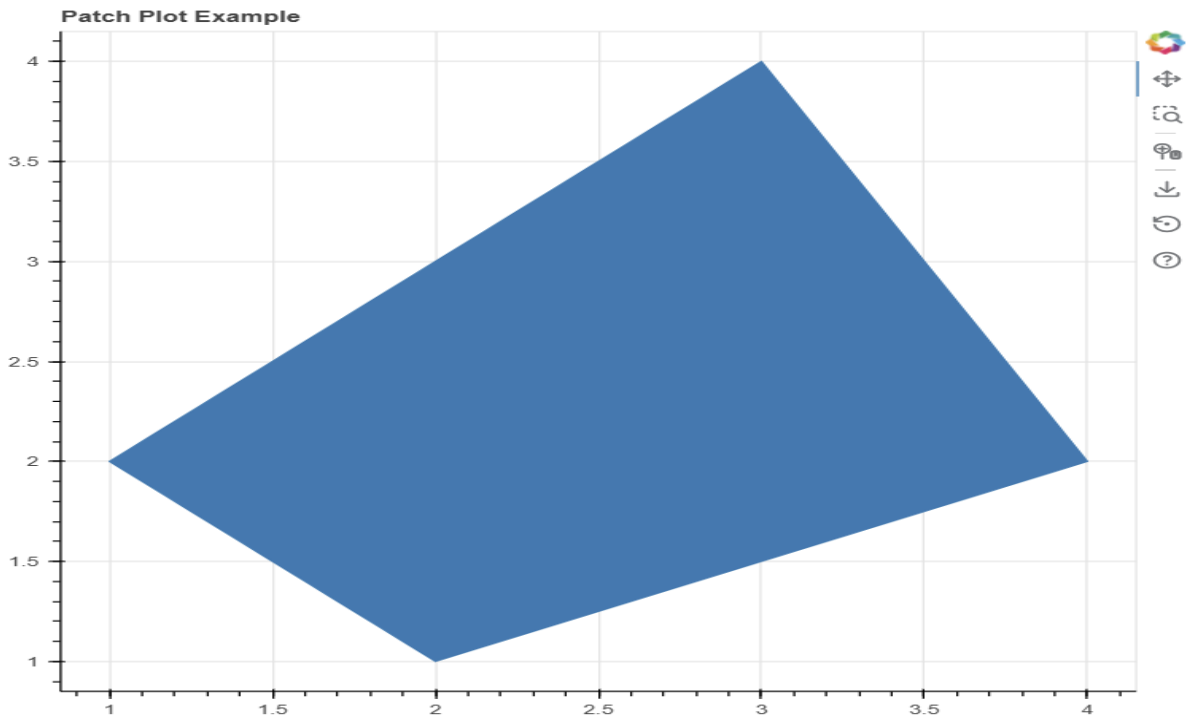
Key Features:

- Represents data as filled polygons
- Customizable shapes using coordinate points
- Can display multiple patches in one plot
- Interactive (zoom, pan, hover)

Code snippet:

```
from bokeh.plotting import figure, show
x = [1, 3, 4, 2]
y = [2, 4, 2, 1]
p = figure(title="Patch Plot Example")
p.patch(x, y)
show(p)
```

Output:



PANDAS

Pandas is a powerful data analysis library in Python that also provides basic data visualization capabilities. It is mainly used for handling and analyzing structured data, but it includes built-in plotting functions that make it easy to create graphs directly from datasets.

Pandas visualization is built on top of Matplotlib, which means it uses Matplotlib internally while providing a simpler interface.

Features of Pandas Visualization

- Simple and quick plotting
- Works directly with DataFrames and Series
- Requires less code compared to Matplotlib
- Suitable for basic data analysis and exploration
- Integrates well with other libraries

Variety of graphs in pandas:

1. Basic Plots

These are the most commonly used graphs:

- Line Plot (df.plot())
- Bar Chart (df.plot.bar())
- Horizontal Bar Chart (df.plot.barh())

2. Statistical Plots

Used for analyzing data distribution:

- Histogram (df.plot.hist())
- Box Plot (df.plot.box())
- Density Plot / KDE (df.plot.kde() or df.plot.density())

3. Relationship Plots

Used to show relationships between variables:

- Scatter Plot (df.plot.scatter())
- Hexbin Plot (df.plot.hexbin())

4. Area and Composition Plots

- Area Plot (df.plot.area())
- Pie Chart (df.plot.pie())

5. Advanced / Less Common Plots

- Line subplots (multiple plots in one figure)
- Stacked plots (bar/area with stacking)

Types of Plots in Pandas:

1. LINE PLOT:

A line plot in Pandas is used to visualize how data changes over a continuous range, such as time or ordered values. It is the default plot type in Pandas and is commonly used to display trends and patterns in data.

Pandas line plots are simple to create because they work directly with DataFrames and Series, without needing much additional code. Internally, Pandas uses Matplotlib to generate the graph.

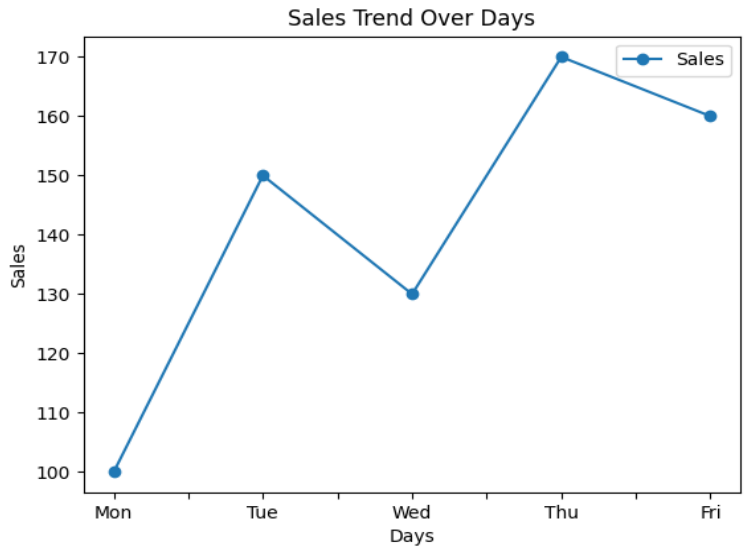
Key Features:

- Displays continuous data trends
- Supports plotting multiple columns as separate lines
- Automatically uses DataFrame index as x-axis (if not specified)
- Easy integration with datasets

Code snippet:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    "Days": ["Mon", "Tue", "Wed", "Thu", "Fri"],
    "Sales": [100, 150, 130, 170, 160]
}
df = pd.DataFrame(data)
df.plot(x="Days", y="Sales", kind="line", marker='o')
plt.title("Sales Trend Over Days")
plt.xlabel("Days")
plt.ylabel("Sales")
plt.show()
```

Output:



2. HISTOGRAM:

A histogram in Pandas is used to visualize the distribution of numerical data by dividing it into intervals called bins. It shows how frequently data values fall within each range, making it useful for understanding the spread, shape, and concentration of the dataset.

Pandas histograms are easy to create directly from a DataFrame or Series, and they internally use Matplotlib for plotting.

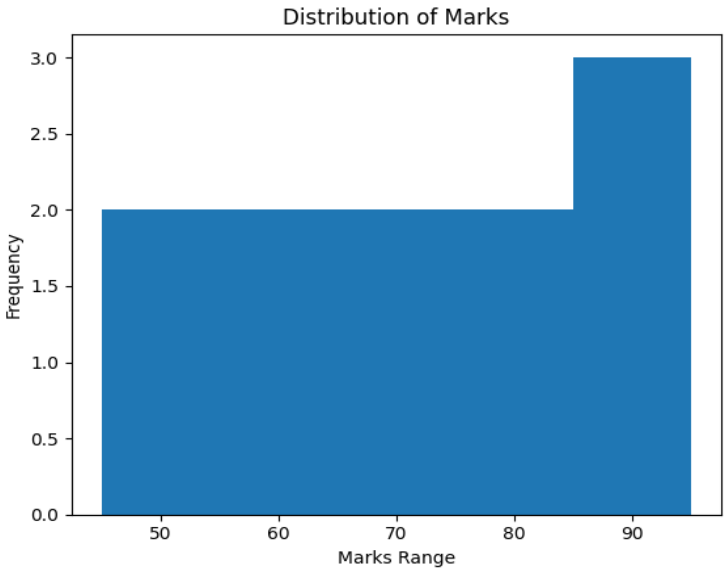
Key Features:

- Displays frequency distribution of data
- Helps identify skewness and spread
- Useful for detecting outliers and patterns
- Can plot multiple columns at once

Code snippet:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    "Marks": [45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]
}
df = pd.DataFrame(data)
df["Marks"].plot(kind="hist", bins=5)
plt.title("Distribution of Marks")
plt.xlabel("Marks Range")
plt.ylabel("Frequency")
plt.show()
```

Output:



3. SCATTER PLOT:

A scatter plot in Pandas is used to display the relationship between two numerical variables. Each point on the graph represents a pair of values, showing how one variable changes with respect to another.

It is mainly used to:

- Identify correlation between variables
- Detect patterns or trends
- Find outliers in the data

Pandas makes it easy to create scatter plots directly from a DataFrame using built-in plotting functions.

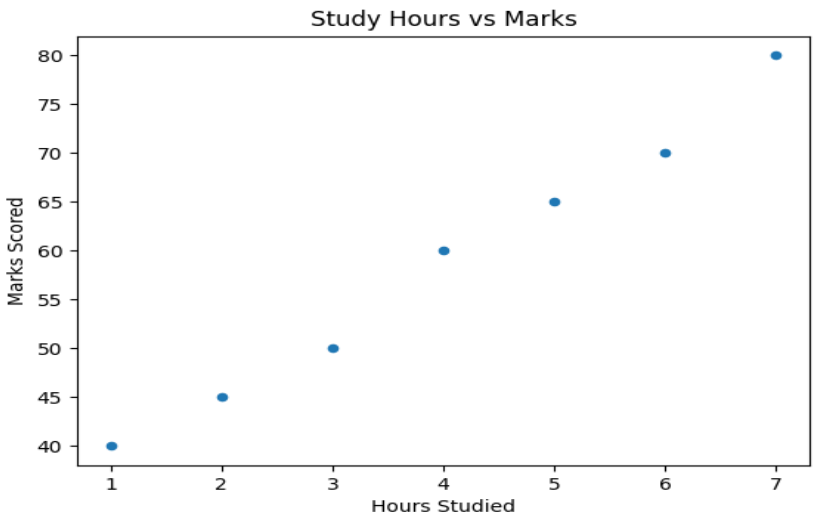
Key Features:

- Displays individual data points
- Helps analyze relationships between variables
- Useful for identifying clusters and outliers
- Simple syntax using DataFrame

Code snippet:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    "Hours_Studied": [1, 2, 3, 4, 5, 6, 7],
    "Marks_Scored": [40, 45, 50, 60, 65, 70, 80]
}
df = pd.DataFrame(data)
df.plot(kind="scatter", x="Hours_Studied", y="Marks_Scored")
plt.title("Study Hours vs Marks")
plt.xlabel("Hours Studied")
plt.ylabel("Marks Scored")
plt.show()
```

Output:



4. PIE CHART:

A pie plot in Pandas is used to represent data as proportions of a whole in the form of a circular chart. Each slice of the pie corresponds to a category, and its size indicates the percentage contribution of that category to the total.

Pie plots are useful when:

- You want to show relative proportions
- Comparing parts of a whole
- Displaying percentage distribution clearly

Pandas allows easy creation of pie charts directly from a Series or DataFrame column.

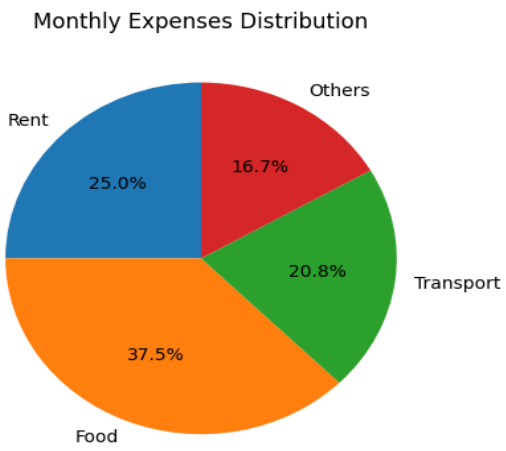
Key Features:

- Circular representation of data
- Shows percentage distribution
- Easy to understand and interpret
- Works well for small datasets

Code snippet;

```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.Series(
    [300, 450, 250, 200],
    index=["Rent", "Food", "Transport", "Others"]
)
data.plot(kind="pie", autopct="%1.1f%%", startangle=90)
plt.title("Monthly Expenses Distribution")
plt.ylabel("")
plt.show()
```

Output:



5. STACKED PLOT:

A stacked plot in Pandas is used to display multiple data series in a way where they are stacked on top of each other. This helps in visualizing the combined total as well as the contribution of each category.

Stacked plots are commonly used in:

- Comparing multiple variables over time
- Showing how individual components contribute to a total
- Visualizing cumulative data

Pandas supports stacked plots mainly in:

- Stacked bar charts
- Stacked area plots

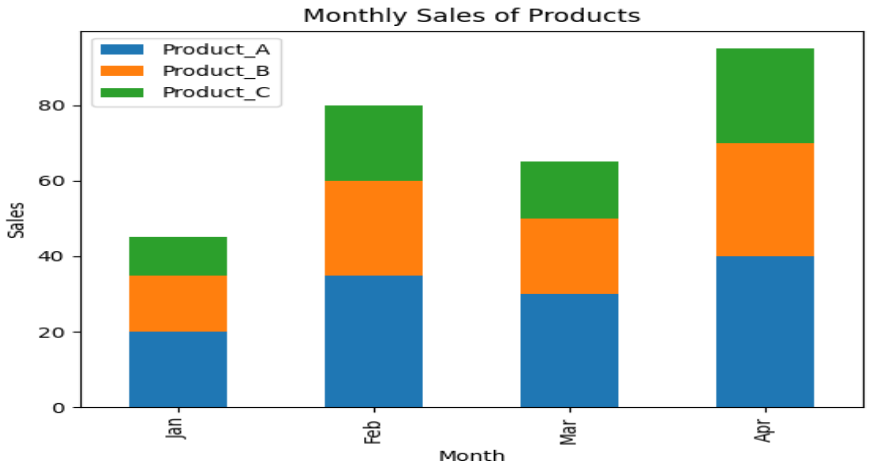
Key Features:

- Displays multiple datasets in one graph
- Shows individual contribution and total value
- Easy comparison between categories
- Useful for trend analysis

Code snippet:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr"],
    "Product_A": [20, 35, 30, 40],
    "Product_B": [15, 25, 20, 30],
    "Product_C": [10, 20, 15, 25]
}
df = pd.DataFrame(data)
df.set_index("Month", inplace=True)
df.plot(kind="bar", stacked=True)
plt.title("Monthly Sales of Products")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

Output:



Comparison for python libraries

Feature	Matplotlib	Seaborn	Plotly	Bokeh	Pandas
Ease of Use	Moderate (requires more code)	Easy (high-level functions)	Easy to Moderate	Moderate	Very Easy
Customization	Very High (full control)	Moderate (limited compared to Matplotlib)	High	High	Low
Interactivity	Very Limited (mostly static)	Limited (static, built on Matplotlib)	Very High (interactive by default)	Very High (interactive tools)	Very Limited
Performance (Large Data)	Good	Good	Moderate (can slow with large data)	Good (optimized for streaming)	Moderate
Graph Variety	Very Wide	Wide (statistical focus)	Very Wide	Wide	Limited
Visual Appearance	Basic (needs styling)	Attractive by default	Highly attractive	Good	Basic
Best Use Case	Custom and detailed plots	Statistical analysis	Interactive dashboards	Real-time and web apps	Quick data analysis
Learning Curve	Medium	Easy	Medium	Medium to High	Very Easy